

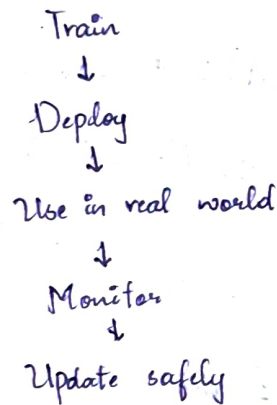
MLOps + Serving + FASTAPI + DOCKER + MLFLOW + DVC + CI/CD +

MONITORING

1. MLOps - The practice of managing the complete machine-learning lifecycle so models can be trained, deployed, monitored, versioned and updated reliably.

Think of it as : $ML + \text{Software Engineering} + \text{DevOps} = \text{MLOps}$

The real goal is :



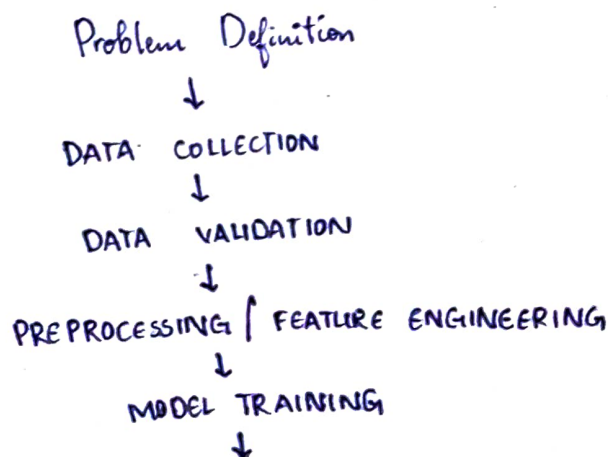
→ Why needed?

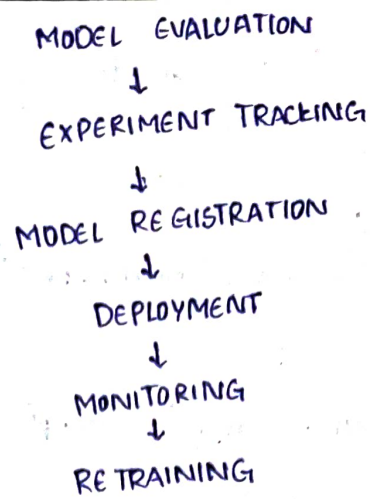
* A model working in a notebook is not enough.

Real-world problems include:

- Data changes
- Model performance drops
- Different model versions
- Dependency issue
- Deployment failures
- Reproducibility problems
- Need for retraining
- " " monitoring

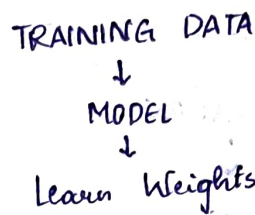
→ ML Lifecycle - is the end-to-end process from raw data to deployed model





→ Training Vs Inference

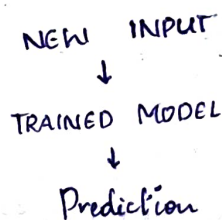
↓
The process where the model learns patterns from data.



Usually: • Expensive

- Happens less often
- Needs large datasets
- Uses GPUs sometimes

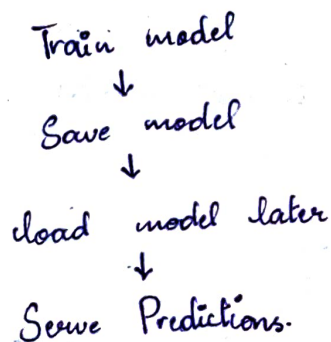
Inference - Using a trained model to predict from new data.



example: training → train spam classifier.
inference → classify a new email.

→ Model Artifact - The saved trained model file used later for inference.

- examples:
- | | |
|----------------|---------------|
| ① model.pkl | ④ model.keras |
| ② model.joblib | ⑤ model.pt |
| ③ model.h5 | ⑥ model.onnx |



→ Model Serialisation - Saving a trained model into a file.

example using joblib:

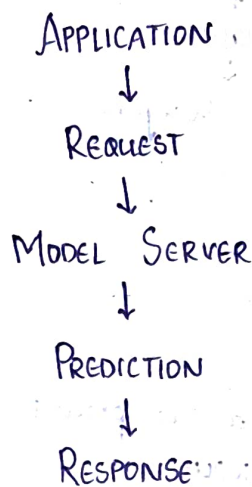
```
→ import joblib
   joblib.dump(model, "model.pkl")
```

Load it:

```
→ model = joblib.load("model.pkl")
```

Why? We do need to retrain the model everytime the API starts.

2. Model Serving - Making a trained model available so applications or users can send input and receive predictions.

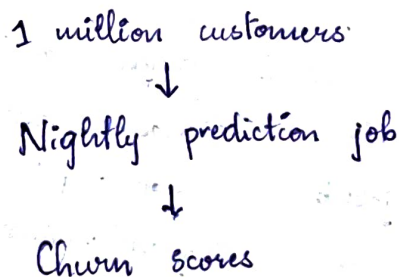


Common serving methods:

- REST API
- Batch prediction
- Streaming
- Embedded model

→ Batch Inference - Predictions are generated for many records together.

example:



Use when:

- Real-time answer is not needed
- Large datasets
- Scheduled processing

Churn - When a customer stops doing business/subscribing with a

company, more like customer attrition.

examples for b.i :

- Daily recommendations
- Monthly risk scoring
- Large report generation

→ Real-time Inference - The model responds immediately to incoming requests.

USER REQUEST



API



MODEL



PREDICTION



USER

examples -

- Fraud detection
- Recommendation API
- Chatbots
- Image prediction

* Metric : Latency

BATCH

1. Many predictions together.
2. Higher delay acceptable.
3. Scheduled job
4. Cheaper in many cases.

REAL - TIME

1. Prediction per request
2. Low latency required.
3. Always available service
4. More infrastructure.

→ Latency - Time taken to process one request and return a result

example: req → 120ms → response

Lower latency is usually better for real-time systems.

→ Throughput - Number of requests the system can process in given time.

example: 300 requests / second

latency → speed of one request

throughput → number of requests handled

→ REST API - A web interface that allows applications to communicate using HTTP requests.

Common HTTP methods:

GET
POST
PUT
DELETE

For ML prediction APIs, we often use:

POST - because we send input data.

FASTAPI

3 FastAPI - A modern Python framework for building APIs quickly.

Why popular for ML means ↓

- Fast
- Automatic API documentation
- Simple
- Supports validation
- Python-based
- Easy to connect to ML models.

→ Basic FastAPI app

→ from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def home():

return {"message": "API running"}

Run: `unicorn main:app --reload`

→ API Endpoint - A URL where an API accepts requests.

example: / (or) /predict

→ @app.post("/predict")

def predict():

.. .. .

→ Request and Response

Request:

{ "age": 25,

"salary": 50000
}

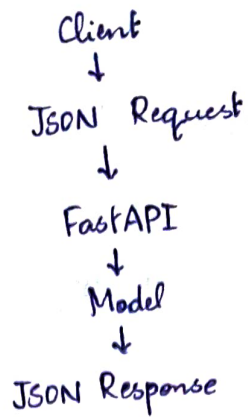
Response:

{

"prediction": 1

}

Flow -



→ Pydantic - A Python library used by FastAPI to validate incoming data.

example:

```
→ from pydantic import BaseModel
class InputData(BaseModel):
    age: float
    salary: float
```

Then:

```
→ @app.post("/predict")
def predict(data: InputData):
```

Why useful?

- Checks data types
- Rejects invalid input
- Makes APIs safer

ML Model with FastAPI

```
→ import joblib
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
model = joblib.load("model.pkl")
```

```
class InputData(BaseModel):
```

```
    feature1: float
```

```
    feature2: float
```

```
@app.post("/predict")
```

```
def predict(data: InputData):
```

```
    x = [[data.feature1, data.feature2]]
```

```
    pred = model.predict(x)
```

```
return {
```

```
    "prediction": int(pred[0])
```

```
}
```

→ Swagger Docs - FastAPI automatically provides documentation

Usually: /docs

This lets you:

- View endpoints
- Test requests
- See schemas

DOCKER

4. Docker - A platform that packages an application and all its dependencies into a portable container.

→ Docker Image - A packaged blueprint containing the application, dependencies and environment.

Assume: Image → template

→ Docker Container - A running instance of a Docker image.

Assume: Container → running application

→ Dockerfile - A file containing instructions for building a Docker image

Example:

```
→ FROM python:3.11
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0",  
     "--port", "8000"]
```

→ Important Dockerfile commands

FROM - Selects base image

→ FROM python:3.11

WORKDIR - Sets working directory

→ WORKDIR /app

COPY - copies files

→ COPY . .

RUN - Executes commands during image build

→ RUN pip install -r requirements.txt

CMD - Specifies what runs when container starts

→ CMD [...]

→ Build Docker Image

→ docker build -t ml-api .

Meaning:

docker build → build image

-t ml-api → image name

→ Run Container

→ docker run -p 8000:8000 ml-api

Port mapping:

Host 8000

↓

Container 8000

→ Docker Workflow

Python App

↓

Dockerfile

↓

docker build

↓

Docker Image

↓

docker run

↓

Container

→ requirements.txt - Stores python dependencies.

→ Environment Variables - External configuration values used instead of hardcoding settings.

examples: API_KEY

DATABASE_URL

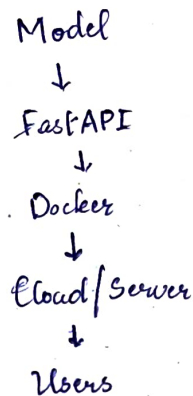
MODEL_PATH

** Do not hardcode secrets inside code or Github repositories

DEPLOYMENT

→ Deployment - Making an application available for real users.

Typical ML deployment -



→ Local vs Production

Local - localhost:8000

Only your machine

Production - hosted on:

• Cloud VM

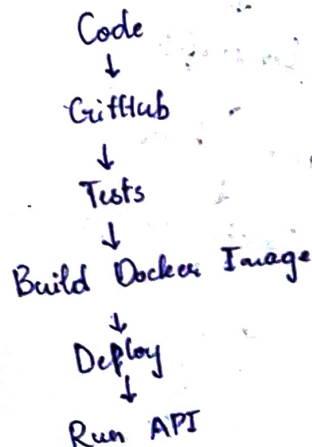
• Kubernetes

• Container service

• Managed ML platform

• Server

Deployment Pipeline



Later CI/CD automates this

MLFLOW

5. MLflow - A platform for tracking machine-learning experiments, models, metrics, parameters, and artifacts.

Many experiments



MLflow



Compare results

Why?

Assume:

Model A

learning-rate = 0.01
accuracy = 91%

Model B

learning-rate = 0.001
accuracy = 94%

Model C

learning-rate = 0.1
accuracy = 85%

Without tracking: Which one was best?

MLflow stores everything.

→ MLflow Run - One experiment execution

Each run may contain:

- Parameters
- Metrics
- Model
- Files
- Metadata

→ MLflow Parameters - Configuration values used during training.
examples: learning-rate, batch-size, n_estimators, max_depth

Log: → `mlflow.log-param("learning-rate", 0.01)`

→ MLflow Metrics - Numerical performance results.

examples: accuracy, precision, recall, F1, RMSE, loss

Log: → `mlflow.log-metric("accuracy", 0.94)`

→ MLflow Artifacts - Files produced during an experiment.

examples: Model files, Graphs, Confusion matrix, Feature importance plots.

Log: → `mlflow.log-artifact("confusion-matrix.png")`

→ Basic Example -

→ import mlflow

with mlflow.start_run():

mlflow.log_param("max-depth", 5)

model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)

mlflow.log_metric("accuracy",
accuracy)

→ Model Registry - A centralized place to manage model versions and lifecycle stages.

example:

Model	v1
"	v2
"	v3

Then:

Development
Staging
Production
Archiving

} terminology can differ by
platform/version

DVC

- 6. DVC (Data Version Control) - A tool used to version large datasets and ML artifacts while integrating with Git.

Git is useful for Code

DVC is useful for:

- Large datasets
- Models
- ML artifacts

Why? Imagine:

dataset.csv
↓
modified
↓
modified again
↓
which version trained model v2?

DVC helps connect:

Code version

+

Dataset version

+

Model version

→ Basic DVC Workflow

1. Initialize:

→ dvc init

2. Track data:

→ dvc add data.csv

This creates something like: data.csv.dvc

3. Commit metadata:

→ git add data.csv.dvc

git commit -m "track dataset"

→ DVC Remote Storage - Large files are usually stored in remote storage.

examples:

• S3

• Google Cloud Storage

• Azure

• SSH

• Local remote

* Git → stores small metadata

Remote Storage → stores large files

→ DVC Pull and Push

Upload: → dvc push

Download: → dvc pull

→ Git Vs DVC

Git

- Code versioning
- Great for small files
- Stores repository history

DVC

- Data/model versioning
- Better for large artifacts
- Tracks large-file versions

They usually work together.

CI/CD

7. CI/CD - Automation that tests, builds, and deploys software whenever changes are made.

CI - Continuous Integration

Automatically:

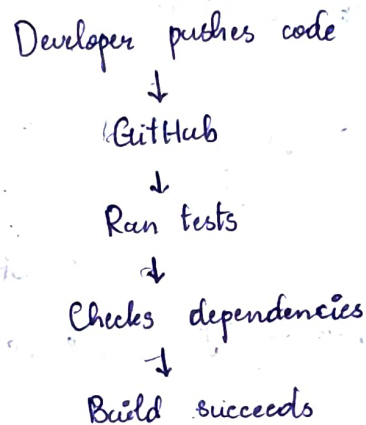
- Run tests
- Check code
- Validate builds

CD - Continuous Delivery / Deployment

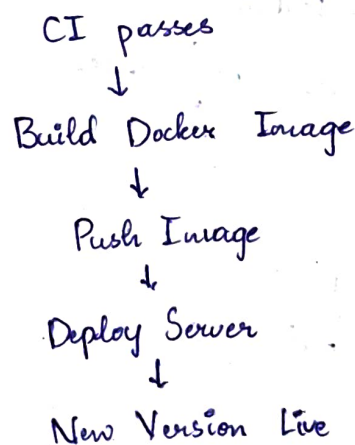
Automates release / deployment steps.

→ CI Pipeline

example:



→ CD Pipeline



→ ML CI/CD

ML adds extra checks.

Code Tests + Data validation + Model tests + Deployment tests

Possible checks:

- Data schema
- Model accuracy threshold

- API tests
- Model file exists
- Docker builds

→ GitHub Actions - A tool that can run CI/CD workflows when code changes.

Workflow file usually: `.github/workflows/`

• Example trigger:

→ YAML
on:

push:

branches:

- main

16/09/2026

MONITORING

• Model Monitoring - Tracking model and system behavior after deployment.

WHAT should we monitor?

Four important groups:

- System Metrics
- Data Metrics
- Model "
- Business "

→ System Metrics - Monitor infrastructure

- examples:
- Latency
 - CPU usage
 - Memory
 - Throughput
 - GPU usage
 - Request count
 - Error rate

→ Data Monitoring - Monitor incoming data.

Check:

- Missing values
- New categories
- Changed distribution
- Invalid values
- Outliers

example:

Training ages: 20-60

Production input: ages 150, 200

Something is wrong.

→ Data drift - The distribution of input features changes over time.

example:

Model trained on % Customers aged mostly 20-40

Later production data % " " " 50-70

$$\text{So, } P_{\text{train}}(X) \neq P_{\text{production}}(X)$$

→ Concept drift - The relationship between inputs and target changes

example:

Before:

High income → low loan default

Later due to economic changes:

High income → default behavior also changes

Conceptually, $P(Y|X)$ changes

→ Prediction drift - Distribution of model predictions changes unexpectedly.

example:

Normally, 10% fraud predictions

Suddenly, 70% fraud predictions

Could indicate:

- Data drift
- Model issue
- Real-world change

→ Model Performance Monitoring - When true labels eventually become available, monitor:

• Accuracy

• Precision

• Recall

• F1

• ROC-AUC

• RMSE

example: initial accuracy : 94%
after 6 months : 78%

Model may need retraining.

→ Error Rate - Percentage of requests that fail.

example: 500 requests
25 failed
Error Rate = 5%

→ Logging - Recording events from the application.

examples:

- Prediction request received
- Model loaded
- Error occurred
- Response completed

Logs help debug production problems.

→ Retraining - Training the model again using newer data.

Possible triggers:

- Performance below threshold
- Data drift detected
- New data accumulated
- Scheduled retraining

→ Model Versioning - Never overwrite models blindly

Use versions: model-v1, model-v2, model-v3

This allows:

- Comparison
- Rollback
- Reproducibility

→ Rollback - Returning to a previous stable model version if the new deployment fails.

→ A/B Testing - Different users receive predictions from different model versions to compare real-world performance.

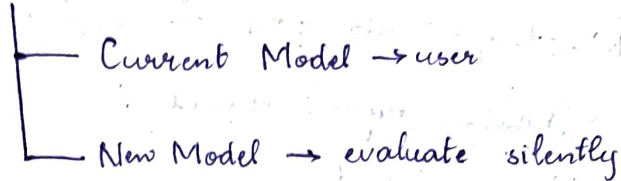
example: 50% users $\rightarrow$ Model A
50% users $\rightarrow$ Model B

Compare:

- Conversion
- Engagement
- Accuracy
- Revenue

$\rightarrow$ Shadow Deployment - A new model receives real production traffic in parallel but its predictions are not shown to users.

Request



$\rightarrow$ Canary Deployment - Deploy new version to a small percentage of users first.

example: 95% $\rightarrow$ old model
5% $\rightarrow$ new model

If stable: increase gradually

MLOps Tool Map

Git $\rightarrow$ code versioning

DVC $\rightarrow$ data/model versioning

MLflow $\rightarrow$ experiment tracking + model management

FastAPI $\rightarrow$ model API

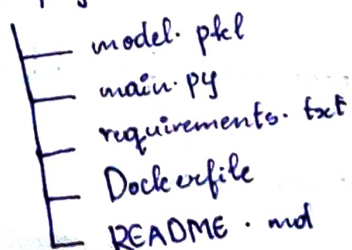
Docker $\rightarrow$ package application

GitHub Actions $\rightarrow$ CI/CD

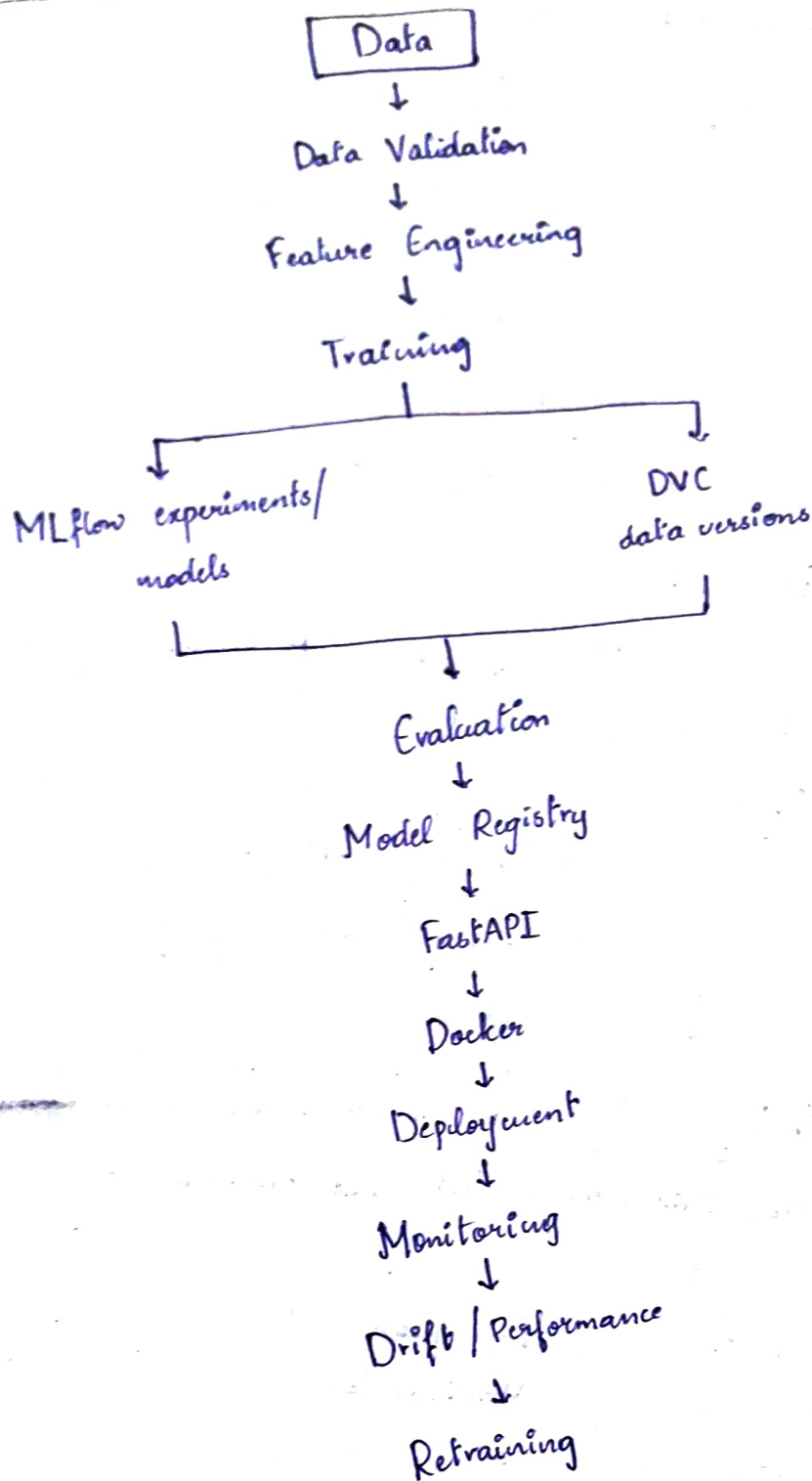
Monitoring $\rightarrow$ production health

$\rightarrow$ Practical Project Folder

ml-project/



→ End-to-End MLOps Pipeline



→ Deployment Failure Examples

Real-world failures can come from:

- Missing dependency
- Wrong model path
- Different preprocessing
- Wrong feature order
- Invalid input
- Memory limits
- Port configuration

◦ Secret / API key problems.

This is why reproducibility matters.

→ Training - Serving Skew - When preprocessing during production differs from preprocessing used during training.

example:

Training - StandardScaler

Production - No Scaling

Model predictions become wrong.

Best Pipeline Practice:

Save preprocessing pipeline + model together

For sklearn:

```
*** Pipeline ([  
    ("scaler", StandardScaler()),  
    ("model", LogisticRegression()),  
])
```

17/09/2026

→ Reproducibility - Ability to reproduce the same ~~experiment~~ using the same code, data, parameters and environment.

Requires tracking:

Code version

Dependencies

Data version

Model version

Hyperparameters

Random seed

→ Model Registry vs Experiment Tracking

Experiment Tracking answers:

Which training run performed best?

Model Registry answers:

Which model version is approved/deployed?

→ DVC → dataset versions

MLflow Registry → model versions.

→ Monitoring without Labels - Sometimes production labels are not immediately available.

Then monitor:

- Input drift
- Prediction distribution
- Confidence scores
- System health

When labels arrive:

- Accuracy
- Precision
- Recall
- RMSE

→ Confidence Monitoring

Example:

Initially: Avg confidence: 0.92

Later: 0.55

May indicate data distribution changed.

→ Model Endpoint Health Check

Production APIs often include something like:

```
→ @app.get("/health")
def health():
    return {"status": "ok"}
```

→ Prediction Endpoint

```
→ @app.post("/predict")
def predict(data: InputData)
    ...
```

* /health → system status

* /predict → model inference

API Status Codes

200 → Success

400 → Bad Request

404 → Not found

500 → Server Error

Model loading Best practice

→ model = joblib.load("model.pkl")

Loads once when application starts.

@app.post("/predict") :

def predict(...):

model = joblib.load("...")

This reloads model for every request.

→ Docker Vs Virtual Environment

V.E - isolates python packages.

Docker - Isolates :

- Application
- Dependencies
- OS-level environment

→ Docker Container

- light weight
- Shares host kernel
- Starts quickly
- Smaller

Virtual Machine

- Heavy
- Own OS
- Slower
- Larger

→ Flask = older, minimal web framework

FastAPI = modern, async-capable, validation + automatic docs

→ MLflow

1. Experiment tracking
2. Metrics
3. Parameters
4. Model management

DVC

1. Data versioning
2. Dataset versions
3. Large artifacts
4. Reproducible data pipeline

→ CI/CD focuses on automating software integration and delivery.

MLOps include CI/CD plus:

- Data management
- Training
- Experiment tracking
- Model versioning
- Monitoring
- Retraining